

PHASE 1 RESEARCH PROPOSAL

Evaluating Hysteresis Aware Adaptive DNN Split Inference under Fluctuating Edge Cloud Network Conditions

Approved research direction Edge Cloud Intelligence

Group information	Details
Group ID	G 31
Section	A B
Primary Researcher	Zohaib Hassan (22i-0800)
Co Researcher	Fahad Saleem (22i-0764)
Topic status	Proposed

Proposal Summary

This study will evaluate whether a hysteresis-aware runtime policy can select a deep neural network split point across an edge node and a cloud node without reacting excessively to short-lived network changes. The policy will be tested against edge-only, cloud-only, fixed-split, and greedy adaptive baselines under steady, step-change, bursty, and periodic network conditions. The study will emphasize tail latency, service-level objective violations, data transfer, adaptation delay, and switching overhead rather than reporting only average latency.

1 Problem and Motivation

Resource-constrained edge devices can provide low-latency and privacy-preserving inference, but they may not have enough compute capacity to execute modern deep neural networks efficiently. Sending every input to a cloud service provides greater compute capacity, yet end-to-end response time becomes sensitive to bandwidth, round-trip time, congestion, and packet loss. Split inference offers a third option: early model layers run at the edge, an intermediate activation is transmitted, and the remaining layers run in the cloud.

A fixed split point can perform well under the condition for which it was selected and poorly after the network changes. A purely greedy adaptive controller can also be unstable: if it moves the split whenever a newly predicted option is marginally better, noisy measurements or oscillating bandwidth may cause repeated switching. Those switches add control and warm-up overhead and can worsen tail latency even when mean latency appears acceptable. A stable adaptive policy is therefore relevant to real-time edge-cloud applications with explicit latency objectives.

2 Initial Research Gap

Prior work established the value of layer-level partitioning, distributed inference, and adaptation to changing bandwidth and resource conditions [1]-[5]. More recent work has improved profiling and latency estimation for distributed inference [6]. However, the initial literature scan suggests that fewer studies isolate the stability cost of runtime repartitioning under rapidly fluctuating or periodic network traces. In particular, there is room for a reproducible comparison that reports tail latency, SLO violations, switch frequency, and adaptation delay while varying the hysteresis threshold and dwell time. This is an initial gap and will be validated and refined through the systematic literature review in Cutoff 2; it is not yet a claim of final novelty.

3 Primary Research Question

Under which patterns and rates of network change does a hysteresis-aware adaptive DNN partitioning policy reduce P95 inference latency and SLO violations relative to edge-only, cloud-only, fixed-split, and greedy adaptive baselines without excessive repartitioning overhead?

4 Research Objectives

1. Build a reproducible edge-cloud split-inference testbed for two image-classification DNNs and expose candidate partition points at model stage boundaries.
2. Measure how steady, step-change, bursty, and periodic network conditions alter the best execution location and split point.
3. Design a lightweight hysteresis-aware policy that changes the split only when predicted benefit exceeds a configurable threshold for a minimum dwell period.
4. Compare all policies using latency distributions, SLO violations, throughput, transmitted data, adaptation delay, switching rate, and controller overhead with repeated runs and uncertainty intervals.

5 Proposed Methodology

5.1 Research Design

The study will use a controlled, repeated-measures experimental design. Each execution policy will process the same request traces under the same emulated network profile. Independent variables will be policy, DNN model, network

pattern, network severity, and edge compute load. Dependent variables will be the metrics defined in Section 6.

5.2 Testbed and Workload

Element	Planned implementation
Edge node	Resource-limited Linux device or Docker container with fixed CPU and memory limits
Cloud node	Linux cloud VM or a separately constrained server
Software	container running the remaining model stages Python, PyTorch, Docker, a small HTTP or gRPC inference service, and Linux traffic control for network emulation
Models	MobileNetV2 and ResNet18 adapted for CIFAR-10; candidate splits placed at major stage boundaries
Dataset	CIFAR-10 test images replayed as a deterministic request stream; labels used only for correctness checks
Request trace	Constant-rate requests plus short bursts; fixed seeds and timestamped configuration files

5.3 Policies and Baselines

Policy	Purpose	Decision rule
Edge only	Local baseline	All layers execute on the edge node
Cloud only	Remote baseline	Input is transmitted and all layers execute in the cloud
Fixed split	Static partition baseline	One offline-selected split remains unchanged during a run
Greedy adaptive	Dynamic baseline	Choose the currently predicted lowest-latency split at every decision interval
Hysteresis aware	Proposed policy	Switch only after improvement exceeds threshold H for K consecutive intervals and minimum dwell time D has elapsed

5.4 Network Scenarios

Scenario	Pattern	Illustrative conditions
Steady	Stable reference	Low, medium, and high bandwidth with fixed RTT
Step change	Abrupt transition	Bandwidth or RTT changes once during the run

Scenario	Pattern	Illustrative conditions
Bursty	Short impairment	Brief bandwidth collapse, delay spike, or 1-3 percent loss
Periodic	Oscillation	Alternating good and poor states at controlled periods

5.5 Experimental Procedure and Analysis

1. Profile edge and cloud execution time and intermediate activation size for each candidate split after a warm-up period.
2. Calibrate representative network profiles, then preserve every profile as a version-controlled script and configuration file.
3. Run each model-policy-scenario combination for a fixed 60-second trace, with ten independent repetitions and randomized execution order where practical.
4. Record per-request timestamps, selected split, measured bandwidth and RTT, bytes transferred, resource utilization, and policy decisions.
5. Report median, P95, and P99 latency; SLO violation rate; throughput; switch count; adaptation delay; controller overhead; and 95 percent bootstrap confidence intervals. Use paired comparisons because policies receive matched traces.
6. Publish code, configurations, seeds, raw logs or compact reproducible result files, and figure-generation scripts in the public repository.

6 Expected Metrics and Outcomes

Metric	Definition and expected analytical use
End-to-end latency	Median, P95, and P99 milliseconds from request creation to prediction response
SLO violation rate	Percentage of requests exceeding a predeclared latency threshold
Throughput	Successfully completed inferences per second
Communication volume	Megabytes transmitted per inference and per trace
Switching rate	Partition changes per minute; identifies controller thrashing
Adaptation delay	Time from a sustained network change to selection of the new stable split
Controller overhead	Decision time and monitoring overhead per interval

Metric	Definition and expected analytical use
Correctness check	Agreement of split predictions with the unsplit reference model within numerical tolerance

Expected outcome. The study is expected to produce a performance map showing when edge-only, cloud-only, static splitting, or adaptive splitting is preferable. It will identify whether hysteresis improves tail latency and SLO compliance by suppressing unnecessary switches, and it will quantify the responsiveness-stability trade-off across threshold and dwell-time settings. These are hypotheses to test; the proposal does not assume that the proposed policy will win in every condition.

7 Risks and Threats to Validity

Risk	Effect	Mitigation
Emulated network	May not capture every wireless behavior	State this limitation and validate a subset over a real LAN or cloud link if available
Limited hardware	May restrict generalization	Use two contrasting model architectures and publish exact resource limits
Timing noise	Can distort tail metrics	Warm up models, randomize run order, repeat runs, and report confidence intervals
Large experiment grid	May exceed the schedule	Pilot first, then retain a small set of discriminating scenarios

7.1 Feasibility and Scope Control

The minimum viable study uses two containerized nodes on one Linux-capable computer with explicit CPU and memory limits. A Raspberry Pi, second laptop, or low-cost cloud VM can be added for external validation but is not required for the core comparison. The experimental grid will be reduced after a pilot study so that only conditions that distinguish policy behavior are retained.

A practical target is five representative 60-second network traces, two models, five policies, and ten repetitions, for approximately 500 measured runs. Automation will make these runs feasible across several days while preserving logs, seeds, and configurations.

7.2 Planned Division of Work

Role	Primary responsibility	Shared responsibility
Primary Researcher	Controller design, experimental orchestration, and repository integration	Literature review, analysis, manuscript, and defense
Co Researcher	Model partitioning, workload generation, and measurement pipeline	Literature review, analysis, manuscript, and defense

8 Initial Academic References

- [1] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, “Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge,” in Proceedings of ASPLOS, 2017, pp. 615-629. doi: 10.1145/3037697.3037698.
- [2] S. Teerapittayanon, B. McDanel, and H. T. Kung, “Distributed Deep Neural Networks over the Cloud, the Edge and End Devices,” in Proceedings of the IEEE International Conference on Distributed Computing Systems, 2017, pp. 328-339. doi: 10.1109/ICDCS.2017.226.
- [3] E. Li, L. Zeng, Z. Zhou, and X. Chen, “Edge AI: On-Demand Accelerating Deep Neural Network Inference via Edge Computing,” IEEE Transactions on Wireless Communications, vol. 19, no. 1, pp. 447-457, 2020. doi: 10.1109/TWC.2019.2946140.
- [4] F. McNamee, S. Dustdar, P. Kilpatrick, W. Shi, I. T. A. Spence, and B. Varghese, “The Case for Adaptive Deep Neural Networks in Edge Computing,” in Proceedings of the IEEE International Conference on Cloud Computing, 2021, pp. 43-52. doi: 10.1109/CLOUD53861.2021.00017.
- [5] G. Liu, F. Dai, X. Xu, X. Fu, W. Dou, N. Kumar, and M. Bilal, “An Adaptive DNN Inference Acceleration Framework with End-Edge-Cloud Collaborative Computing,” Future Generation Computer Systems, vol. 140, pp. 422-435, 2023. doi: 10.1016/j.future.2022.10.033.
- [6] R. Viramontes and A. Davoodi, “DIME: Distributed Inference Model Estimation for Minimizing Profiled Latency,” in Proceedings of the IEEE International Conference on Smart Computing, 2024, pp. 356-361. doi: 10.1109/SMART-COMP61445.2024.00081.
- [7] D. Rosendo, A. Costan, P. Valduriez, and G. Antoniu, “Distributed Intelligence on the Edge-to-Cloud Continuum: A Systematic Literature Review,” Journal of Parallel and Distributed Computing, vol. 166, pp. 71-94, 2022. doi: 10.1016/j.jpdc.2022.04.004.

Reference Verification Links

<u>10.1145/3037697.3037698</u>		<u>10.1109/ICDCS.2017.226</u>		<u>10.1109/TWC.2019.2946140</u>
	<u>10.1109/CLOUD53861.2021.00017</u>		<u>10.1016/j.future.2022.10.033</u>	
<u>10.1109/SMARTCOMP61445.2024.00081</u>		<u>10.1016/j.jpdc.2022.04.004</u>		